Name: **CSE 450, Fall 2026 – Quiz 2A (Parsing)**

Each question below is worth 1 point, for 10 points total. Choose one answer for each multiple-choice question. For question 8 you must write a grammar. Questions 9 and 10 concern your submitted Project 1; correct answers will vary by student and will be graded on whether your answers accurately describe your submission, not what the "best" implementation would be.

1. Which grammar gives '*' higher precedence than '+', with both operators LEFT associative? Use EXPR2 as the start symbol and NUM as a numeric-literal token.

Hint: Check the precedence levels and which side of each rule is recursive.

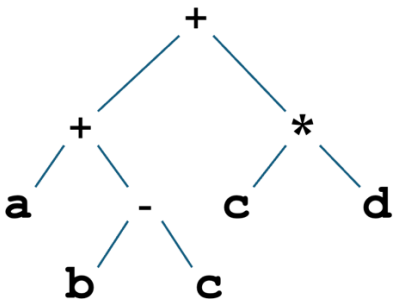
A.
```
EXPR2 -> EXPR2 '+' EXPR1 | EXPR1
EXPR1 -> EXPR1 '*' TERM | TERM
TERM  -> NUM | '(' EXPR2 ')'
```

B.
```
EXPR2 -> EXPR1 '+' EXPR2 | EXPR1
EXPR1 -> TERM '*' EXPR1 | TERM
TERM  -> NUM | '(' EXPR2 ')'
```

C.
```
EXPR2 -> EXPR2 '*' EXPR1 | EXPR1
EXPR1 -> EXPR1 '+' TERM | TERM
TERM  -> NUM | '(' EXPR2 ')'
```

D.
```
EXPR2 -> EXPR1 '*' EXPR2 | EXPR1
EXPR1 -> TERM '+' EXPR1 | TERM
TERM  -> NUM | '(' EXPR2 ')'
```

2. Using standard precedence and associativity, which expression would produce the following AST?



A. `a + b - c + c * d`

B. `a + (b - c) + c * d`

C. `a + ((b - c) + c * d)`

D. `(a + b) - c + c * d`

3. Given the expression:

```
result = var1 * var2 - var3 / (var4 + var5)
```

Which calculation is completed FIRST when the abstract syntax tree (AST) is executed? Assume standard precedence and associativity, and evaluate each binary operator's left subtree completely before its right subtree.

Hint: Start at the root. Evaluate each operator only after evaluating its operands.

A. `result = var1`

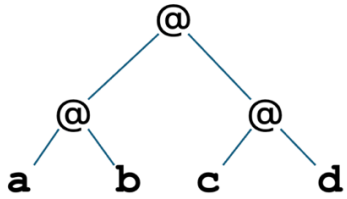
B. `var1 * var2`

C. `var2 - var3`

D. `var4 + var5`

E. ASTs with this set of operations cannot be executed.

4. If the operator @ is LEFT associative, which expression would produce this AST?



A. `a @ b @ c @ d`

B. `(a @ b) @ c @ d`

C. `a @ (b @ c) @ d`

D. `a @ b @ (c @ d)`

E. It's not possible to create this tree with left associativity.

5. In an expression AST that represents operations and operands, what role do grouping parentheses play?

A. They determine how nodes are connected during parsing. Once that structure is built, no separate parenthesis nodes or special execution rules are needed.

B. They add annotations to an AST node, indicating that the child subtrees should be executed in a different order from how they are listed.

C. They add "relay" nodes to the AST that just pass along values from their own children, without altering those values, but their presence is critical to the correct execution of the tree.

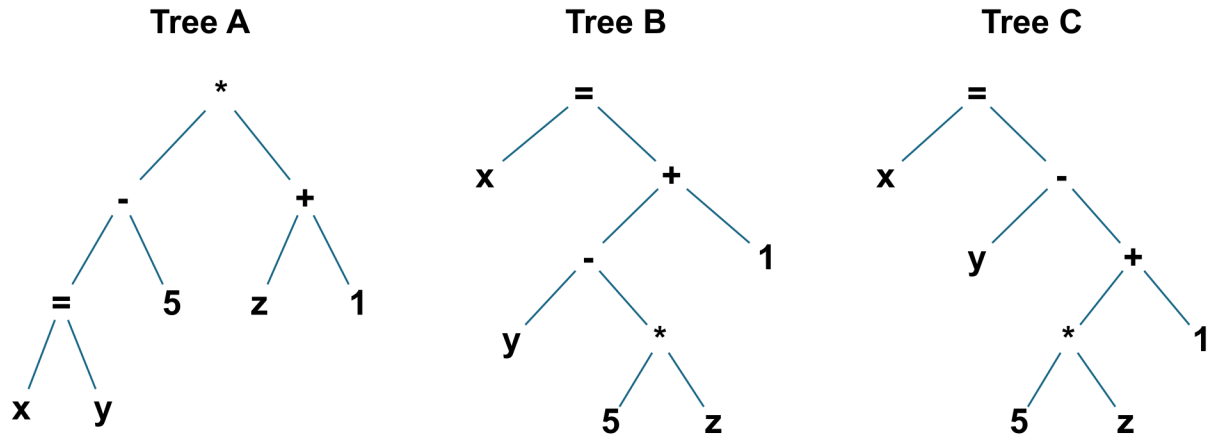
D. They create "loopback" nodes that allow portions of the AST to be rerun based on the placement of parentheses.

E. They are used to flag "high priority" nodes that should be executed first, potentially different from the order that they are stored in the tree.

6. Given the expression:

```
x = y - 5 * z + 1;
```

Which of the following ASTs would be produced using standard precedence and associativity, before any optimization?



***Hint**: With y = 20 and z = 3, the expression assigns 6 to x. Check each tree's structure and result.*

A. Tree A only

B. Tree B only

C. Tree C only

D. More than one of these trees are legal possibilities

E. None of the above trees

7. How should a standard WHILE-loop AST node be structured and executed? Assume zero is false, nonzero is true, and the body has no break or return statements.

A. It should have one subtree representing the *body*. Execute it by repeatedly running the body until it returns a zero, then finish.

B. It should have two subtrees: one representing the *condition* and one representing the *body*. Execute by running a loop: execute the condition and then execute the body. If the body ever returns a zero, exit the loop and finish.

C. It should have two subtrees: one representing the *condition* and one representing the *body*. Execute it by repeatedly running the condition; if it returns a non-zero result, execute the body and then continue the loop. Otherwise finish.

D. It should have three subtrees: a *left condition*, *right condition* and a *body*. Execute it by repeatedly executing the left condition and the right condition, checking that they return identical results before executing the body. If the conditions ever differ, finish.

E. It should have three subtrees: a *condition*, a *body*, and an *alternative*. Execute it by repeatedly running the condition; if it returns a non-zero value, run the body, otherwise run the alternative. If either the body or the alternative return a zero, finish.

8. Rewrite the following grammar to make it unambiguous and make ^ RIGHT associative:

```
S -> S '^' S
   | '(' S ')'
   | NUM
```

S is the start symbol, and NUM is a numeric-literal token. Preserve the same language, including parenthesized expressions. You may introduce new nonterminals.

9. Which answer best describes how your submitted lexer distinguished the seven Marquee command words (LOAD, SAVE, JOIN, CUT, REPEAT, WAIT, and SHOW)?

A. It produced **one common token type** for ALL keywords and the interpreter examined the token's lexeme to determine the specific command.

B. It produced **seven distinct token types**, one for each command.

C. It produced **NO keyword-specific token types**, instead emitting all command words as ordinary variable/identifier tokens, and the interpreter compared their text manually.

D. It removed command words during preprocessing in the lexer and returned a **command number** (0 through 6) that was used to index into a function table.

E. It used completely different approach, or command recognition was not implemented at all.

10. Which answer best describes how your submitted Project 1 represented the Marquee value stack?

A. You used a container of type `std::array<std::string, N>` for the value stack. (N is a fixed capacity.)

B. You used a container of type `std::stack<std::string>` for the value stack.

C. You used a container of type `std::vector<std::string>` for the value stack.

D. You used another container or a custom data structure for the value stack.

E. You **did not implement** the value stack.